DEPARTMENT OF CSE, CS&IT, AI&DS **2400033271 V.Deepthi**

COURSE CODE: 24SDCS02 / 24SDCS02E / 24SDCS02P / 24SDCS02L

FULL STACK APPLICATION DEVELOPMENT

Date of the Session: _/_/_____ Time of The Session: ___to_____

#SKILL - 3 Working with HQL - Sorting, Pagination & Aggregates

Prerequisites:

- Basic knowledge of Hibernate
- Basic idea of HQL syntax

In this lab, you will extend the Hibernate module created in Skill 2 and perform advanced data retrieval using HQL, including sorting, pagination, filtering, and aggregate functions. These operations help in retrieving structured and meaningful data from the database using Hibernate.

Tasks :

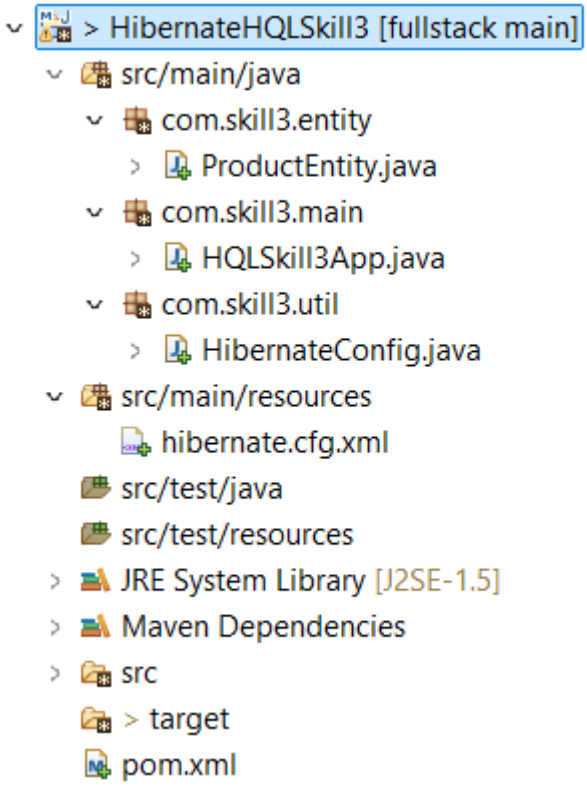
1. Use the Product entity created in Skill 2.
2. Add 5–8 additional Product records into the database.
3. Write HQL queries to retrieve all products sorted by price:
   a. Ascending order
   b. Descending order
4. Write an HQL query to sort products by quantity (highest first).
5. Implement pagination using HQL to display:
   a. First 3 products
   b. Next 3 products
6. Write HQL queries for aggregate operations:
   a. Count total number of products
   b. Count products where quantity > 0
   c. Count products grouped by description
   d. Find minimum and maximum price
7. Write an HQL query using GROUP BY to group products by description.
8. Write an HQL query using WHERE to filter products within a price range.
9. Write HQL queries using LIKE such as:
   a. Names starting with certain letters
   b. Names ending with certain letters

c. Names containing a pattern anywhere (substring)

d. Names with an exact character length

10. Push the Hibernate project to GitHub.

Project Structure:



ProductEntity.java

```
package com.skill3.entity;

import javax.persistence.*;

@Entity

@Table(name = "product")

public class ProductEntity {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private int productId;

    private String productName;

    private double price;
```

```
    private int quantity;
    private String description;
    public ProductEntity() {}
    public ProductEntity(String productName, double price, int quantity, String description) {
        this.productName = productName;
        this.price = price;
        this.quantity = quantity;
        this.description = description;
    }
    public int getProductId() { return productId; }
    public String getProductName() { return productName; }
    public double getPrice() { return price; }
    public int getQuantity() { return quantity; }
    public String getDescription() { return description; }
    public void setProductId(int productId) { this.productId = productId; }
    public void setProductName(String productName) { this.productName = productName; }
    public void setPrice(double price) { this.price = price; }
    public void setQuantity(int quantity) { this.quantity = quantity; }
    public void setDescription(String description) { this.description = description; }
}
```

HQLSkillApp.java

```
package com.skill3.main;
import com.skill3.entity.ProductEntity;
import com.skill3.util.HibernateConfig;
import org.hibernate.Session;
import org.hibernate.Transaction;
import org.hibernate.query.Query;
import java.util.List;
public class HQLSkill3App {
```

```
public static void main(String[] args) {
    Session session = HibernateConfig.getSessionFactory().openSession();
    Transaction tx = session.beginTransaction();
    session.save(new ProductEntity("Laptop", 55000, 10, "Electronics"));
    session.save(new ProductEntity("Mobile", 25000, 15, "Electronics"));
    session.save(new ProductEntity("Keyboard", 1200, 25, "Accessories"));
    session.save(new ProductEntity("Monitor", 15000, 5, "Electronics"));
    session.save(new ProductEntity("Tablet", 18000, 8, "Electronics"));
    tx.commit();
    Query<Object[]> q1 = session.createQuery(
        "select p.description, count(p) from ProductEntity p group by p.description",
        Object[].class
    );
    List<Object[]> groupList = q1.list();
    for (Object[] o : groupList) {
      System.out.println(o[0] + " : " + o[1]);
    }
      Query<ProductEntity> q2 = session.createQuery(
        "from ProductEntity p where p.price between :min and :max",
        ProductEntity.
);
    q2.setParameter("min", 1000.0);
    q2.setParameter("max", 30000.0);
    q2.list(
    session.createQuery(
        "from ProductEntity p where p.productName like 'M%'",
        ProductEntity.class
    ).list(
    session.createQuery(
        "from ProductEntity p where p.productName like '%e'",
        ProductEntity.class
```

```
        ).list();
        session.createQuery(
                "from ProductEntity p where p.productName like '%top%'",
                ProductEntity.class
        ).list();
        session.createQuery(
                "from ProductEntity p where length(p.productName) = 5",
                ProductEntity.class
        ).list();
        session.close();
        HibernateConfig.getSessionFactory().close();
    }
}
```

HibernateConfig.java

```
package com.skill3.util;
import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;
public class HibernateConfig {
    private static SessionFactory factory;
    static {
        factory = new Configuration().configure().buildSessionFactory();
    }
    public static SessionFactory getSessionFactory() {
        return factory;
    }
}
```

Hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
```

```
    "https://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>
    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/hibernate_db
    </property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">12345</property>
    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQL8Dialect
    </property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <mapping class="com.skill3.entity.ProductEntity"/>
  </session-factory>
</hibernate-configuration>
```

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-
4.0.0.xsd">
 <modelVersion>4.0.0</modelVersion>
 <groupId>com.skill3</groupId>
 <artifactId>HibernateHQLSkill3</artifactId>
 <version>0.0.1-SNAPSHOT</version>
   <dependencies>
 <dependency>
        <groupId>org.hibernate</groupId>
```

```
        <artifactId>hibernate-core</artifactId>
        <version>5.6.15.Final</version>
    </dependency>
    <dependency>
        <groupId>javax.persistence</groupId>
        <artifactId>javax.persistence-api</artifactId>
        <version>2.2</version>
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.33</version>
    </dependency>
    </dependencies>
</project>
```

Outputs:

```
[Java Application] C:\Users\vasav\Downloads\sts-4.18.0.RELEASE\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_17.0.6.v20230204-1729\jre\bin\javaw.exe  (18-Feb-2026, 9:19:04 pm – 9:19:11 pm) [pid: 28
Feb 18, 2026 9:19:11 PM org.hibernate.engine.transaction.jta.platform.internal.JtaPlatformInitiator initiateService
INFO: HHH000490: Using JtaPlatform implementation: [org.hibernate.engine.transaction.jta.platform.internal.NoJtaPlatform]
Hibernate: insert into product (description, price, productName, quantity) values (?, ?, ?, ?)
Hibernate: insert into product (description, price, productName, quantity) values (?, ?, ?, ?)
Hibernate: insert into product (description, price, productName, quantity) values (?, ?, ?, ?)
Hibernate: insert into product (description, price, productName, quantity) values (?, ?, ?, ?)
Hibernate: insert into product (description, price, productName, quantity) values (?, ?, ?, ?)
Hibernate: select productent0_.description as col_0_0_, count(productent0_.productId) as col_1_0_ from product productent0_ group by product
Electronics : 23
Accessories : 8
Furniture : 2
Hibernate: select productent0_.productId as producti1_0_, productent0_.description as descript2_0_, productent0_.price as price3_0_, product
Hibernate: select productent0_.productId as producti1_0_, productent0_.description as descript2_0_, productent0_.price as price3_0_, product
Hibernate: select productent0_.productId as producti1_0_, productent0_.description as descript2_0_, productent0_.price as price3_0_, product
Hibernate: select productent0_.productId as producti1_0_, productent0_.description as descript2_0_, productent0_.price as price3_0_, product
Hibernate: select productent0_.productId as producti1_0_, productent0_.description as descript2_0_, productent0_.price as price3_0_, product
```

```
:scription

:oductName as productn4_0_, productent0_.quantity as quantity5_0_ from product productent0_ where productent0_.price between ? and ?
:oductName as productn4_0_, productent0_.quantity as quantity5_0_ from product productent0_ where productent0_.productName like 'M%'
:oductName as productn4_0_, productent0_.quantity as quantity5_0_ from product productent0_ where productent0_.productName like '%e'
:oductName as productn4_0_, productent0_.quantity as quantity5_0_ from product productent0_ where productent0_.productName like '%top%'
:oductName as productn4_0_, productent0_.quantity as quantity5_0_ from product productent0_ where length(productent0_.productName)=5
```

VIVA QUESTIONS:

1. **What is HQL and how does it differ from SQL?**
2. **Why do we use sorting in HQL?**
3. **What is pagination and where is it used?**
4. **Name any three aggregate functions supported in HQL.**
5. **What is the difference between HQL and HCQL (Hibernate Criteria Query Language)?**

1ANS: HQL is Hibernate Query Language that works with Java objects, not tables.

Unlike SQL, it is database-independent and object-oriented.

2Ans: To arrange query results in ascending (ASC) or descending (DESC) order for easier viewing and reporting.

3Ans: Pagination splits results into smaller pages**.**

Used in web apps to display limited records per page and improve performance.

4Ans: count(), sum(), avg()

5Ans: HQL: String-based query for static queries.

HCQL (Criteria API): Object-oriented, dynamic queries built in Java.

(For Evaluator's use only)

Comment of the Evaluator (if Any)	Evaluator's Observation Marks Secured: out of _ EMP ID & Full Name of the Evaluator: Signature of the Evaluator Date of Evaluation: